

Zadanie 10

Natalia Włodyka

Grudzień 27, 2025

1 Polecenie zadania

Celem zadania jest skonstruowanie naturalnego splajnu kubicznego dla funkcji i węzłów z zadania 08.

2 Zadane funkcja

Zadana w zadaniu 08 funkcja dla $x \in [-1.25, 1.25]$:

$$f(x) = \frac{1}{1 + 5x^2}$$

Której węzły znajdują się w punktach $-\frac{7}{8}, -\frac{5}{8}, -\frac{3}{8}, -\frac{1}{8}, \frac{1}{8}, \frac{3}{8}, \frac{5}{8}, \frac{7}{8}$.

3 Układ równiań dla niewiadomych ξ_j

W celu wyliczenia wartości ξ rozwiązujemy układ $A\xi = b$, gdzie macierz $A \in R^{(n-2) \times (n-2)}$ ma postać trójdagonalną:

$$A = \begin{bmatrix} 4 & 1 & 0 & \dots & 0 \\ 1 & 4 & 1 & \dots & \dots \\ 0 & 1 & 4 & \dots & 0 \\ \dots & \dots & \dots & \dots & 1 \\ 0 & \dots & 0 & 1 & 4 \end{bmatrix}$$

Wektor b dany jest wzorem, gdzie h jest stałą równą 2:

$$b_i = \frac{6}{h^2}(f_{i+2} - 2f_{i+1} + f_i), \quad i = 0, 1, 2, \dots, n-3$$

Układ $A\xi = b$ rozwiążemy za pomocą algorytmu Thomasa.

4 Wyznaczanie współczynników splajnu i jego wartości

Wartości splajnu dla dowolnego $x \in [x_j, x_{j+1}]$ wyliczamy ze wzoru:

$$P(x) = Af_j + Bf_{j+1} + C\xi_j + D\xi_{j+1}$$

Gdzie:

$$A = \frac{x_{j+1} - x}{h}, B = \frac{x - x_j}{h}, C = \frac{h^2}{6}(A^3 - A), D = \frac{h^2}{6}(B^3 - B), h = x_{j+1} - x_j$$

5 Kod w Pythonie

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from numpy.typing import NDArray
4
5 vector = NDArray[np.float64]
6 matrix = NDArray[np.float64]
7 number = np.float64
8
9 def thomas_method(sub: vector, diagonal: vector, sup: vector, b:
    vector) -> vector:
10     N = len(sub)
11     cp = np.zeros(N, dtype= number)
12     dp = np.zeros(N, dtype= number)
13     xi = np.zeros(N, dtype= number)
14
15     #Forward
16     cp[0] = sup[0] / diagonal[0]
17     dp[0] = b[0] / diagonal[0]
18     for i in np.arange(1, (N), 1):
19         dnum = diagonal[i] - sub[i] * cp[i - 1]
20         cp[i] = sup[i] / dnum
21         dp[i] = (b[i] - sub[i] * dp[i - 1]) / dnum
22
23     #Back
24     xi[(N - 1)] = dp[N - 1]
25
26     for i in np.arange((N - 2), -1, -1): #
27         xi[i] = (dp[i]) - (cp[i]) * (xi[i + 1])
28
29     return xi
30
31 def function_x(x:vector) -> vector:
32     f_x = np.zeros(len(x), dtype= number)
33     for i in range(len(x)):
34         f_x[i] = 1/(1 + 5*x[i]*x[i])
35
36     return f_x
37
38 def spline_coefficients(x_val: number, x_j: number, h: number) ->
    tuple[number, number, number, number]:
39     A = (x_j + h - x_val) / h
40     B = (x_val - x_j) / h
```

```

41
42 C = (1.0 / 6.0) * (A ** 3 - A) * (h**2)
43 D = (1.0 / 6.0) * (B ** 3 - B) * (h**2)
44 print(A,B,C,D)
45
46 return A, B, C, D
47
48
49 def spline_function(f_x: vector, h: number, x_val : number, x:
vector, xi: vector) -> number:
50     j = np.searchsorted(x, x_val) - 1
51     if j < 0:
52         j = 0
53     if j >= len(x) - 1:
54         j = len(x) - 2
55
56     A, B, C, D = spline_coefficients(x_val, x[j], h)
57
58     P_x = A * f_x[j] + B * f_x[j + 1] + C * xi[j] + D * xi[j + 1]
59     return P_x
60
61
62 def main():
63     x: vector = np.array([-7 / 8, -5 / 8, -3 / 8, -1 / 8, 1 / 8, 3
/ 8, 5 / 8, 7 / 8], dtype=np.float64)
64     f_x = function_x(x)
65     diagonal: vector = np.full(len(x) - 2, 4.0)
66     subdiagonal: vector = np.full(len(x)- 3, 1.0)
67     superdiagonal: vector = np.full(len(x)- 3, 1.0)
68     h = x[1] - x[0]
69     b: vector = np.zeros(len(x) - 2, dtype=number)
70     for i in range(len(x) - 2):
71         b[i] = (6.0 / (h ** 2) * (f_x[i + 2] - 2 * f_x[i + 1] + f_x
[i]))
72     vector_xi_0 = thomas_method(subdiagonal, diagonal,
superdiagonal, b)
73     vector_xi: vector = np.zeros(len(x), dtype=number)
74     vector_xi[1:len(x) - 2] = vector_xi_0
75
76     # Tworzymy wykres
77     range_of_x = np.linspace(-7/8, 7/8, 1000)
78     spline = np.zeros_like(range_of_x)
79     for k, val in enumerate(range_of_x):
80         spline[k] = spline_function(f_x, h, val, x, vector_xi)
81
82     plt.plot(range_of_x, spline, label='Naturalny splajn kubiczny',
color='purple')
83     plt.plot(x, f_x, 'o', label='Wezly interpolacyjne', color='pink
')
84     plt.title('Naturalny splajn kubiczny')
85     plt.grid(True)
86     plt.xlabel('x')
87     plt.ylabel('y')
88     plt.legend()
89     plt.show()
90
91 if __name__ == "__main__":

```

6 Wykresy naturalnego splajnu kubicznego

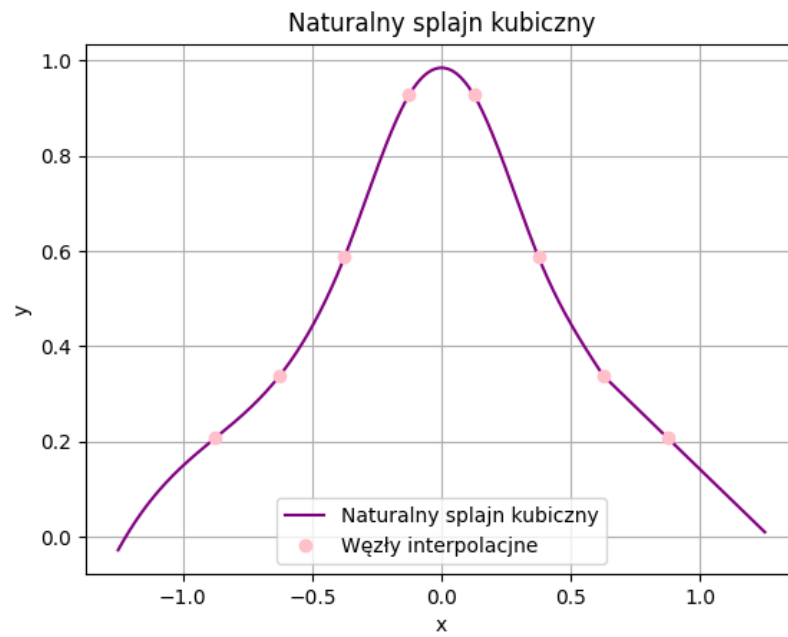


Figure 1: Splajn w przedziale $x \in [-1.25, 1.25]$

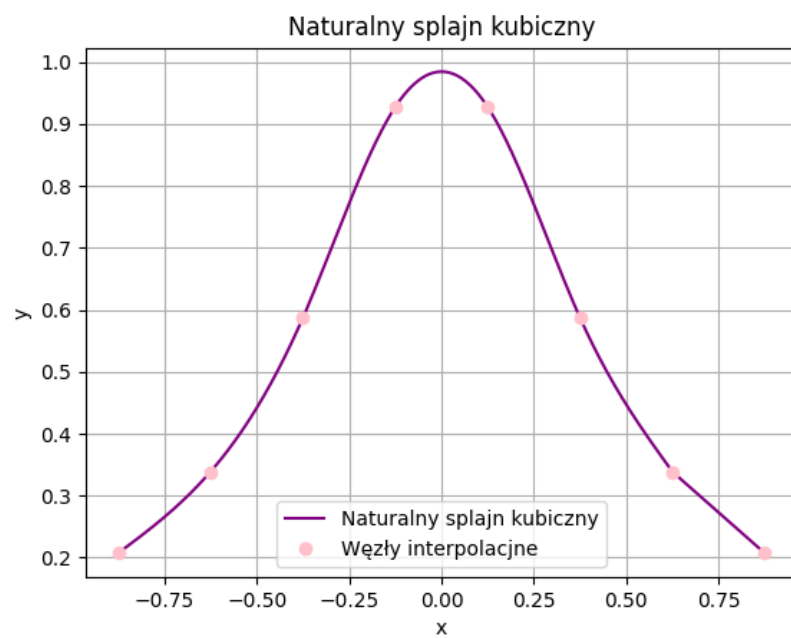


Figure 2: Splajn w przedziale $x \in [-7/8, 7/8]$